

Autonomous Robot Systems Project 1

Group Project Report

A0337935R

A0338361A

A0338562X

March 2, 2026

Abstract

This report presents the implementation and evaluation of autonomous navigation for a TurtleBot 3 in an indoor obstacle course. Three global planning approaches were compared: A* on an octile grid, A* with line-of-sight waypoint skipping as post-processing, and Theta* as an any-angle planner. All three planners were combined with Savitzky–Golay path smoothing and a Regulated Pure Pursuit controller that incorporates curvature-based velocity scaling, obstacle proximity deceleration, and adaptive lookahead distance. The system was developed using ROS 2 and tested both in Gazebo simulation and on physical hardware. Results show that A* with post-processing and Theta* produce qualitatively similar paths that are shorter and smoother than raw A*, which is constrained to octile directions. Analysis of commanded velocity jerk revealed that all planners exhibit high jerk due to abrupt controller transitions, and we demonstrate in simulation that a first-order low-pass filter or rate limiting filter on the velocity commands can roughly halve the RMS jerk while also reducing the total time to goal. Practical challenges encountered during physical experiments, including SLAM drift from the robot getting stuck and network-induced stoppages, are also discussed.

Contents

1	Problem Definition	3
2	Software and Hardware Set Up	3
3	Global Planner	4
3.1	Basic A* Implementation	4
3.2	Postprocessing Techniques	5
3.2.1	Smoothing	5
3.2.2	Waypoint Skipping	6
3.3	Theta* Implementation	7
4	Controller	8
4.1	Regulated Pure Pursuit Implementation	9
4.2	Positioning at Goal	10
5	Parameter Tuning	11
5.1	Tuning Lookahead Gain and Minimum Lookahead Distance	12
5.2	Tuning Proximity Threshold and Max Access Cost	12
5.3	Tuning Curvature Threshold and Finetuning	12
6	Results and Discussion	12
7	Further Improvements	16
8	Contribution	19

1 Problem Definition

The goal of this project was to autonomously navigate a robot from a starting position through an obstacle course, as shown in fig. 1.

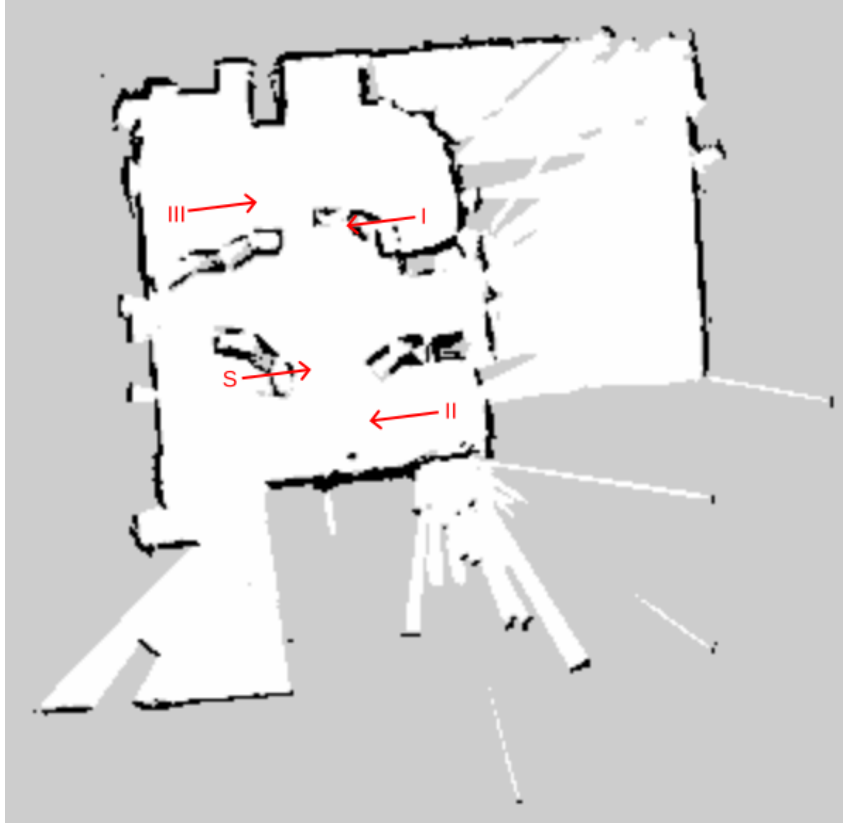


Figure 1: Map generated by the SLAM algorithm, used for path planning. Letters denote positions and arrows indicate orientations. **S** marks the starting position of the robot, while **I**, **II**, and **III** mark the goal positions.

Our task was to adapt the Nav2 Plugins for the planner and controller of the robot such that it could reliably navigate through the arena to each goal position.

2 Software and Hardware Set Up

The used robot is a TurtleBot 3, shown in fig. 2. It is equipped with a 360-degree LiDAR and an onboard gyroscope as well as a Raspberry Pi 5 for computation. The robot was controlled using ROS 2 Jazzy [5], and live telemetry was monitored through Gazebo [4]. The planner and controller were implemented in C++17. The map has been created using already implemented SLAM function which is called during teleoperation of the robot. Chrony was used to achieve correct time synchronisation between the Turtlebots RaspberryPi (client) and the computer running the planner and controller (server).

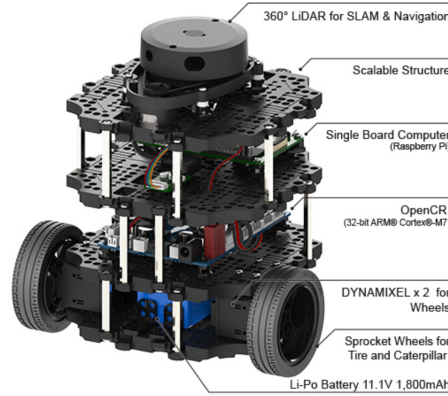


Figure 2: Example image of the Turtlebot 3 from *turtlebot.net.cn*.

3 Global Planner

This section describes the global planning algorithms and postprocessing techniques used to generate a collision-free and optimal path from the robot's current location to the selected goal position.

3.1 Basic A* Implementation

For path planning, an A* algorithm [3] was implemented, as outlined in algorithm 1. The algorithm operates on a discrete grid map in which obstacles are represented through a cost map. This cost map is derived from the SLAM-generated map (fig. 1) by assigning a maximum cost to occupied grid cells and a lower cost to neighbouring cells based on their distance from obstacles, resulting in the cost map shown in fig. 5. Each node in the search graph corresponds to the centre of a grid cell, and nodes are connected to their neighbours in octile direction.

A* maintains an open list of candidate nodes, prioritised by their f -score, which is the sum of the cost-so-far g and a heuristic estimate h of the remaining cost to the goal. At each iteration, the node with the lowest f -score is expanded, and its neighbours are evaluated: if a cheaper path to a neighbour is found, that neighbour's cost and parent are updated and it is added to the open list. The heuristic h must be admissible—that is, it must never overestimate the true cost—to guarantee that A* returns an optimal path. In our implementation, we evaluated both the Euclidean and Chebyshev distance heuristics, both of which are admissible for the octile grid.

Algorithm 1 A* Algorithm

```

1: Initialize all nodes with  $g \leftarrow \infty$ ,  $f \leftarrow \infty$ , parent  $\leftarrow$  null, expanded  $\leftarrow$  false
2:  $g(s_{\text{start}}) \leftarrow 0$ ,  $f(s_{\text{start}}) \leftarrow 0$ 
3: Push  $s_{\text{start}}$  onto open_list
4: while open_list is not empty do
5:    $n \leftarrow$  pop node with smallest  $f$  from open_list
6:   if  $n$  already expanded then
7:     continue
8:   end if
9:   if  $n = s_{\text{goal}}$  then
10:    return reconstructPath( $n$ )
11:   end if
12:   Mark  $n$  as expanded
13:   for each accessible neighbor  $m$  in octile directions do
14:      $\tilde{g} \leftarrow g(n) + \text{dist}(n, m) \times \text{cost}(m)$ 
15:     if  $\tilde{g} < g(m)$  then
16:        $g(m) \leftarrow \tilde{g}$ 
17:        $m.\text{parent} \leftarrow n$ 
18:        $f(m) \leftarrow g(m) + h(m)$ 
19:       if  $m$  not expanded then
20:         Push  $m$  onto open_list
21:       end if
22:     end if
23:   end for
24: end while
25: return empty path

```

3.2 Postprocessing Techniques

After generating the initial path with the basic A* implementation, postprocessing techniques are applied to improve path quality.

3.2.1 Smoothing

The A* algorithm generates a path that is optimal in a grid with octile movement directions which can include sharp turns. It is difficult for a differential drive robot to follow these paths as due to the differential constraints it can only move in the direction of its heading as seen in eq. (2). The jagged path can cause an oscillatory movement in the pure pursuit controller as the lookahead points would change more in their lateral position. Due to this the robot would move more inefficiently. We chose to apply Savitzky Golay smoothing [7] to postprocess the generated path. This algorithm tries to fit a polynomial curve of degree p for a specific number of consecutive waypoints in the path which is defined by the half-window size m . The beginning and end waypoints of the previous path $\hat{p}$ are used multiple times to account for the edges as seen in eq. (5). To calculate the waypoint positions of the smoothed path, a kernel is constructed from the Vandermonde matrix which is shown in eq. (3). The kernel values α_i is taken from the first row of the A matrix calculated given eq. (4) which is then correlated with the previous waypoints

as shown in eq. (5). The parameters proposed by the documentation of the project have been successfully tested in simulation and adopted in the final implementation.

$$\dot{x} = \frac{r(\dot{\phi}_R + \dot{\phi}_L)}{2} \cos \theta \quad (1a)$$

$$\dot{y} = \frac{r(\dot{\phi}_R + \dot{\phi}_L)}{2} \sin \theta \quad (1b)$$

$$\dot{\theta} = \frac{r(\dot{\phi}_R - \dot{\phi}_L)}{L} \quad (1c)$$

$$(1d)$$

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0 \quad (2)$$

$$\mathbf{J} = \begin{bmatrix} 1 & -m & \cdots & (-m)^{p-1} & (-m)^p \\ 1 & -m+1 & \cdots & (-m+1)^{p-1} & (-m+1)^p \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & m-1 & \cdots & (m-1)^{p-1} & (m-1)^p \\ 1 & m & \cdots & m^{p-1} & m^p \end{bmatrix} \quad (3)$$

$$\mathbf{A} = (\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top \quad (4)$$

$$\hat{p}_i = \sum_{j=-m}^m p_k \alpha_{j+m+1}, \quad k = \begin{cases} 1 & \text{if } i+j < 1 \\ n & \text{if } i+j > n \\ i+j & \text{otherwise} \end{cases} \quad (5)$$

3.2.2 Waypoint Skipping

Waypoint skipping was added as a post-processing step to improve the path generated by the octile-grid A* planner. While A* guarantees an optimal path on the discrete grid, that path is still constrained to a limited set of movement directions and is therefore not as short or as taut as a path in continuous space, where the robot can travel at any heading. As a result, the raw path often contains unnecessary intermediate waypoints and small step-like direction changes. Waypoint skipping addresses this by removing intermediate points that are not needed for obstacle avoidance, producing a path that more closely resembles the continuous-space route the robot would ideally follow.

The implemented method uses a line-of-sight (LOS) check between path waypoints. Starting from an anchor waypoint, the algorithm attempts to connect directly to a farther waypoint if the straight-line segment between them remains collision-free. If LOS is satisfied, the intermediate waypoints are removed. In this implementation, a backward search was used: from the current anchor point, the algorithm first attempts to connect to the furthest remaining waypoint, then steps backward until LOS is found. This was chosen over a forward search because it can handle cases where LOS is temporarily blocked and then regained farther along the path after the robot has gone around an obstacle. This

behavior could occasionally be observed in simulation, where the map contained narrower obstacles and more opportunities for visibility to disappear and reappear, although it was less relevant on the physical test map.

During simulated testing, it became clear that aggressive pruning introduced its own problems. If LOS pruning was too permissive, large sections of the path could be reduced to only one or two waypoints around sharp bends, which is particularly problematic near the ends of walls or inflation zones. This caused the local controller to cut corners and approach obstacles too closely. As such, similar issues would be expected during physical testing: a path that is geometrically shorter is not always easier or safer to track. To address this, LOS pruning was made significantly more conservative than the main A* planner. The planner itself uses a relatively high `max_access_cost` so that it can still find feasible routes through narrow spaces when necessary. In contrast, the LOS pruning stage uses a much lower `los_max_access_cost`, so shortcuts are only accepted when they stay well clear of inflated obstacle boundaries. Empirically, this was the key change that caused the path to round corners more naturally instead of drawing overly aggressive straight-line shortcuts.

Two additional implementation choices were made to improve the final result. First, LOS checking was implemented using a Digital Differential Analyzer (DDA)-based grid traversal, which made it straightforward to oversample along the straight-line segment and include corner checks when moving diagonally between cells. Second, after pruning, the path was re-interpolated before Savitzky-Golay smoothing. Interpolation does not change the geometry of the pruned path, but it restores a denser set of waypoints so that the smoothing stage has enough local information to produce a well-behaved curve. This ordering was determined through simulated testing. If smoothing was applied too early, before interpolation, the polynomial fit was not sufficiently constrained by nearby points and could overshoot into inflation zones or even obstacles. The final pipeline was therefore A* planning, LOS waypoint pruning, waypoint interpolation, and then Savitzky-Golay smoothing. This combination preserved the feasibility of the original A* path while producing a cleaner and more trackable trajectory.

3.3 Theta* Implementation

Another way to improve the global path planner performance would be to implement Theta* algorithm [6]. Theta* is an enhanced version of A* that also allows to create paths that are not restricted to 90 ° and 45 °, but it does not guarantee optimality. Compared to A* algorithm, only the part of the neighbor loop changes. Algorithm 2 shows that instead of choosing node n as the parent of the neighbor, it looks whether there is a suitable path to the parent of node n . A suitable path can be found with the Bresenham's Line Algorithm [1] which checks whether the straight line connection between the neighbor node and parent encounters an obstacle. This is done by following the straight line in small steps and checking whether the intermediate points are located in grid cells with a high cost. If a "Line of Sight" (LOS) exists, the parent of the node n is chosen to be the neighbors node parent. It is important to estimate the traversal cost adequately to avoid going too close to obstacles. Figure 3 visualizes our implementation of estimating the total travel cost from the parent to the neighbor node. Additionally to checking whether the cell is accessible when traveling a short distance along the connection, it also increments the total travel cost by the product of the node costs and the short distance. Thereby

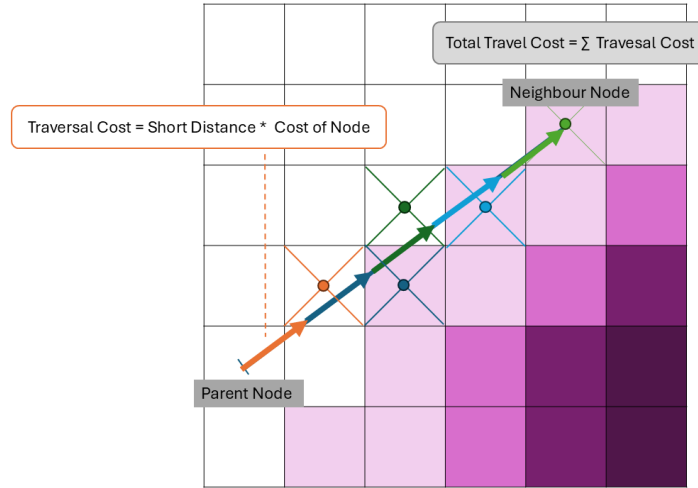


Figure 3: Visualization of "Line of Sight" total travel cost calculation. Darker purple color shows higher value in cost map.

we can assign a higher traversal cost to paths that are closer to physical obstacles and thereby penalize paths that could risk collision. As the assignment of the parent skips intermediate points, it is important to add an interpolation operation before smoothing the entire path in `reconstructPath()`.

Algorithm 2 Theta* Algorithm

```

1: //Initialization and first instructions same as A* algorithm (Line 1 - 12)
2: for each accessible neighbor  $m$  in octile directions do
3:   if  $n.parent \neq \text{null}$  and  $\text{LOS}(n.parent, m)$  then
4:      $\tilde{g} \leftarrow g(n.parent) + \text{LOSCost}(n.parent, m)$ 
5:     if  $\tilde{g} < g(m)$  then
6:        $g(m) \leftarrow \tilde{g}$ 
7:        $m.parent \leftarrow n.parent$ 
8:        $f(m) \leftarrow g(m) + h(m)$ 
9:       if  $m$  not expanded then
10:        Push  $m$  onto open_list
11:       end if
12:       continue
13:     end if
14:   end if
15: //Fallback to standard A* update: path through  $n$  (Line 13 - 23)
16: end for
17: return empty path

```

4 Controller

This section describes the controller used to follow the planned path.

4.1 Regulated Pure Pursuit Implementation

The local path-following controller was implemented using Regulated Pure Pursuit. As in standard pure pursuit, the controller first identifies the point on the global path that is closest to the robot, and then selects a lookahead point farther along the path at a distance of at least L_h . This lookahead point is transformed into the robot frame, giving coordinates (x', y') . Using the geometry of pure pursuit, the required curvature is then computed as

$$c = \frac{2y'}{d^2}, \quad (6)$$

where d is the distance from the robot to the selected lookahead point. The angular velocity command is then based on

$$\omega = vc, \quad (7)$$

where v is the commanded linear velocity. In this way, the robot steers toward a point ahead on the path rather than attempting to track each waypoint directly.

While pure pursuit is simple and effective, by itself it can behave poorly in tight turns or near obstacles. For this reason, three regulated pure pursuit heuristics were added on top of the base controller. First, a curvature heuristic reduces the commanded linear velocity when the required curvature exceeds a chosen threshold. This prevents the robot from entering tight turns too aggressively and improves tracking stability when the path bends sharply. A theoretical starting point for this threshold can be obtained by rearranging eq. (7) as $c_{\text{thresh}} = \omega_{\text{max}}/v_{\text{desired}}$, which gives the curvature at which the commanded angular velocity would begin to saturate for a chosen nominal speed. In practice, however, the final threshold was selected empirically based on observed tracking performance. Second, a proximity heuristic reduces linear velocity when the LiDAR reports that the nearest obstacle is within a specified distance. This gives the robot more time to react in narrow passages and reduces the chance of clipping corners. Third, the lookahead distance was varied proportionally with the robot speed with an additional minimum lookahead distance enforced.

$$L_h = \max(vg_l, L_{h,\min}) \quad (8)$$

This variable lookahead is important because larger lookahead distances at higher speeds produce smoother motion, while smaller lookahead distances at lower speeds improve path-following precision. The lower bound was also important because if the lookahead distance becomes too small, the curvature term in eq. (6) becomes highly sensitive to small changes in y' , which can lead to oscillatory yaw corrections and jitter.

An additional practical issue arose when the lookahead point was located behind the robot. Pure pursuit assumes forward motion toward a target point that is roughly in front of the robot. When the lookahead point lies behind the robot, this assumption breaks down. In the special case where the point is directly behind, y' is close to zero and the curvature can also become close to zero, causing the robot to continue driving forward even though the target is behind it. More generally, if the point is behind but offset to one side, the controller tends to command a large forward turning arc, which can result in non-ideal behaviour such as overshoot or poor convergence to the path. To address this, an additional check was implemented using the relative bearing

$$\theta' = \text{atan2}(y', x'). \quad (9)$$

If $|\theta'| > 4\pi/5$, meaning the lookahead point is strongly behind the robot, the controller suppresses forward motion and commands a rotation in place instead. This behavior was especially useful at the start of navigation when the robot heading differed significantly from the path direction, and when the robot was near a wall, where attempting to turn through a large forward arc would be undesirable or unsafe.

Finally, the commanded velocities were bounded by the robot's physical limits. The angular velocity was clipped to the maximum allowable angular speed, and the linear velocity was clipped to the maximum allowable linear speed. In addition, once the robot came within the goal position threshold, path-following behavior was suspended and control was handed over to a separate goal-alignment routine. This allowed the robot to stop translating and rotate to match the desired final heading at the goal.

4.2 Positioning at Goal

If the threshold in x-y-direction is reached, we switch to a different P-controller that is responsible for turning the robot in the correct position at the goal. Even though the differential constraints allow the robot to rotate in place (by setting one wheel velocity opposite to the other) it could still move away from the goal position due to the uneven ground and motor calibration issues. We saw that after during turning of the robot it left the goal threshold and tried to reach it again, which caused oscillation between the two states. We implemented a Schmitt-Trigger like system that allowed to adapt the threshold for the final goal position and increase it after the goal position was reached such that heading adjustments would not cause the oscillation anymore.

5 Parameter Tuning

Parameter	Cause & Effect	Value
<i>Regulated Pure Pursuit</i>		
Lookahead Gain	Scales lookahead distance with velocity. Too high gain values cause too large lookahead distances which means points nearby will be ignored and the robot is unable to follow the path accurately risking collision. On the other hand, lower values improve precision but can cause oscillation and jittering along the path.	2.0
Proximity Threshold	Distance at which the robot begins decelerating near obstacles. Higher values increase safety margins but reduce speed in cluttered environments.	0.25
Curvature Threshold	Curvature above which the robot reduces speed. Lower values cause earlier deceleration into turns; higher values maintain speed through sharper curves.	1.8
Yaw Gain	P-Controller value for turning the robot in the correct direction at the goal position. High values lead to oscillation and low values cause too slow adjustments. This behavior is similar to controlling a stable LTI system with a linear controller.	0.8
Min. Lookahead dist.	Clipping the lookahead distance to avoid collapsing during slow motion	0.175
<i>Savitzky–Golay Smoothing</i>		
Smoothing Order	Polynomial order of the smoothing filter. Higher order fitting preserves sharp features but may introduce curves; lower order fitting produces smoother but less faithful paths.	3
Half-Window Size	Number of neighboring points used for smoothing. Larger windows produce smoother paths because it is fitted over larger path segments but may deviate from the original trajectory or push the path into obstacles. Smaller windows are closer to the original path and may result in insufficient smoothing.	4
<i>A* Planner</i>		
<code>max_access_cost</code>	Cost threshold above which a cell is treated as inaccessible. Lower values keep paths farther from obstacles; higher values allow paths through narrow gaps and close to obstacles.	200
Planner Frequency	Frequency at which the planner replans the path. The planner should run slower than the controller. Due to the nature of problem (non-convex) successive plans can deviate strongly. Rapid switching between different reference trajectories can cause the controller to perpetually jumping between references and result in oscillatory or unstable behavior.	8.0

Table 1: Tuned Hyperparameters

Tuning parameters introduced in the controller and planner is necessary to guarantee efficient and reliable movements of the Turtlebot. It is important to design real world tests to be able to select good performing hyperparameters. Three test environment have been selected in the parkour which are shown in fig. 4 to tune the parameters associate with Regulated Pure Pursuit and A* Planner. The Savitzky-Golay Smoothing parameters have been taken from the documentation of the project after testing them in simulation.

5.1 Tuning Lookahead Gain and Minimum Lookahead Distance

The first test was to drive a straight line in a free space. We first used a small lookahead gain and increased it until the oscillation along a straight path stopped. With this we could determine the minimum lookahead distance based on the desired speed and the selected lookahead gain.

5.2 Tuning Proximity Threshold and Max Access Cost

For tuning the proximity threshold we chose a small corridor in the parkour as the second test environment. Here we drove a straight line between the two obstacles and modified both the max access cost and proximity threshold. Regarding the max access cost we looked at different values and the resulting path. Our aim was to select a value at which the path can be generated and is not jagged as well as not planned to close to the obstacles. Regarding the proximity threshold we chose a value that did not induce too much slowing down.

5.3 Tuning Curvature Threshold and Finetuning

For the curvature threshold we chose an environment where we anticipated the highest curvature that the robot would drive in the parkour. The third environment is the one at the start position of the robot. Our aim was to select a curvature threshold that enabled the robot to maneuver carefully around the obstacle but did not slow done too much. It was difficult for us to visually evaluate the performance of the robot at different thresholds as the behaviour is coupled to the proximity threshold and lookahead gain, so we used a Foxglove [2] dashboard, seen in fig. 5, to replay recorded MCAP files and plot the curvature, commanded angular velocity, commanded linear velocity and lookahead distance. We additionally plotted the lookahead point and planned path for evaluation in the graphic. With that we could select a suitable curvature threshold based on the extreme values in the curvature plot. We also used this visualization for finetuning the previously found parameters.

6 Results and Discussion

This section will present the results of using the three different planners A*, A* with way-point skipping and Theta* for the obstacles course and evaluate the general performance of the robot. Making general quantitative statements from these experiments is very difficult. Simple comparisons such as time to goal can be misleading, because of different goal locations and noisy SLAM measurements causing different path lengths. Qualitatively, one can quickly see a great improvement from A* to A* with post-processing or Theta*. This can be explained by the graph used for planning where each cell is only connected to

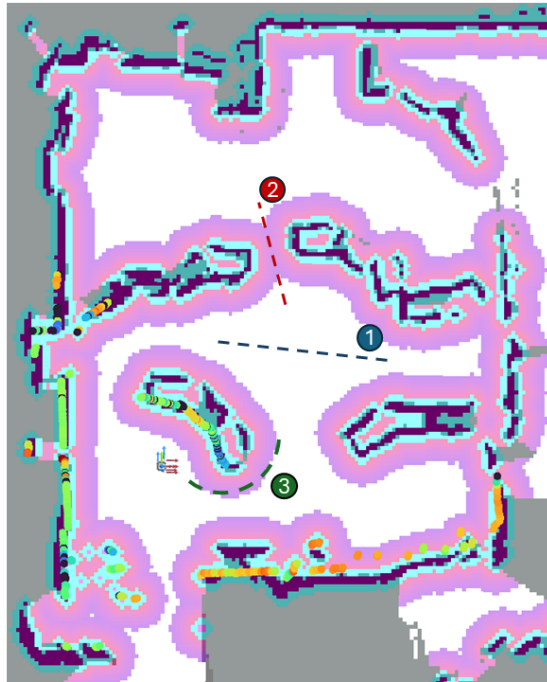


Figure 4: Three test environments for parameter tuning

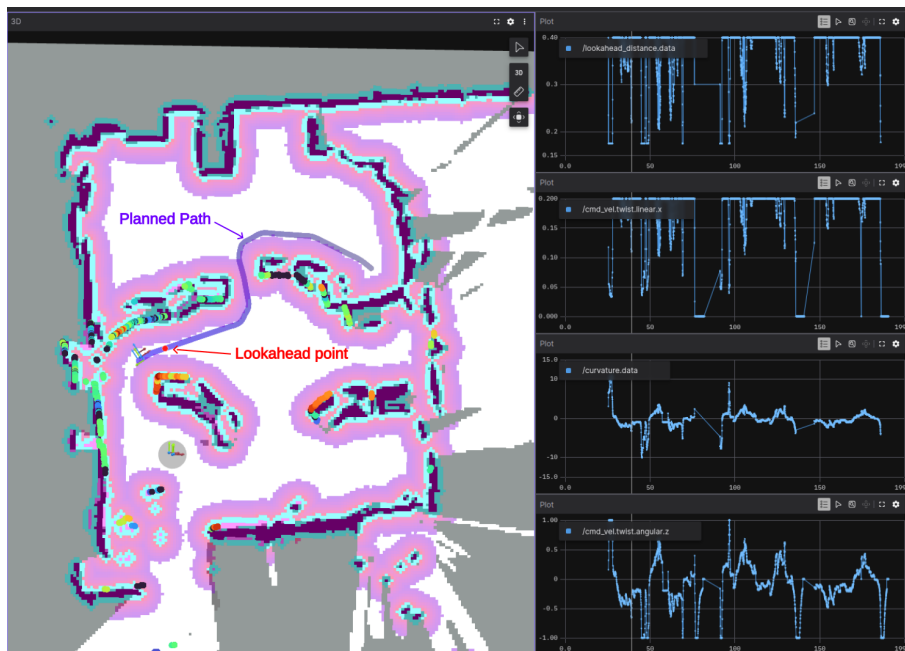


Figure 5: Foxglove dashboard showing on the left the map with **planned path**, the robot and the **lookahead point**. On the right are various metrics plotted used for hyperparameter tuning.

its eight neighbouring cells. A consequence of this discretisation is that the resulting path generated by A^* is constrained to eight directions per cell. Rather than travelling in a straight line at an arbitrary angle, the robot follows a staircase-like trajectory consisting of horizontal, vertical, and 45-degree segments. This pattern persists even after smoothing, as shown in fig. 6. Two heuristics were evaluated—Euclidean distance and Chebyshev distance—both of which yielded similar results. The postprocessing of the A^* path as described in section 3.2.2 could exclude the stair-case path which mostly occurred during transition $I \rightarrow II$ and $II \rightarrow III$. It also reduced the path length compared to the initial A^* algorithm. Qualitatively and in experiments A^* post-processed and Theta* behaved almost identical, but A^* post-processed has the advantage that it is computationally simpler than Theta*.

Although we were able to successfully navigate to each goal position using every planning algorithm, operating the robot in a real environment introduced a number of practical challenges. A major issue encountered was the robot getting stuck. The robot hardware itself was quite primitive as the rear omni wheel frequently got stuck in the small grooves of the floor. When the robot believed it was moving forward but was not actually progressing, the estimated probabilities of nearby obstacles would deviate from the map as seen in fig. 7. There is a mismatch of the probabilities of occupancy resulting from the odometry values and the ones measured with the LIDAR which causes the corridor to close as seen in fig. 7c. In our case we saw that during passage of the corridor the error "No path found" appeared as the passage was blocked with too high cost values to be able to generate a path through it. After the passing the corridor, fig. 7e and fig. 7f show a slow recovery and alignment of localization with further measurements.

As many robots were connected to the same network while filming the final runs using A^* , A^* postprocessing and Theta*, the robot stopped multiple times as seen in the video recordings. This is caused by the congestion of the network which creates a delay in the response between computer and turtlebot up to a specific time out forcing the robot to stop its operation.

The map plays a crucial role for being able to plan a reliable path. As the obstacles are hollow objects (chairs with newspaper on them) no solid boundaries are measured during SLAM, especially for the boundaries. This caused the issue that after the robot got stuck and the localization and mapping issues occurred, it tried to find a path going outside of the arena to the first goal.



Figure 6: Comparison of planned paths across all three segments. Columns show raw A*, A* with post-processing, and Theta*. Rows show the segments $S \rightarrow I$, $I \rightarrow II$, and $II \rightarrow III$.

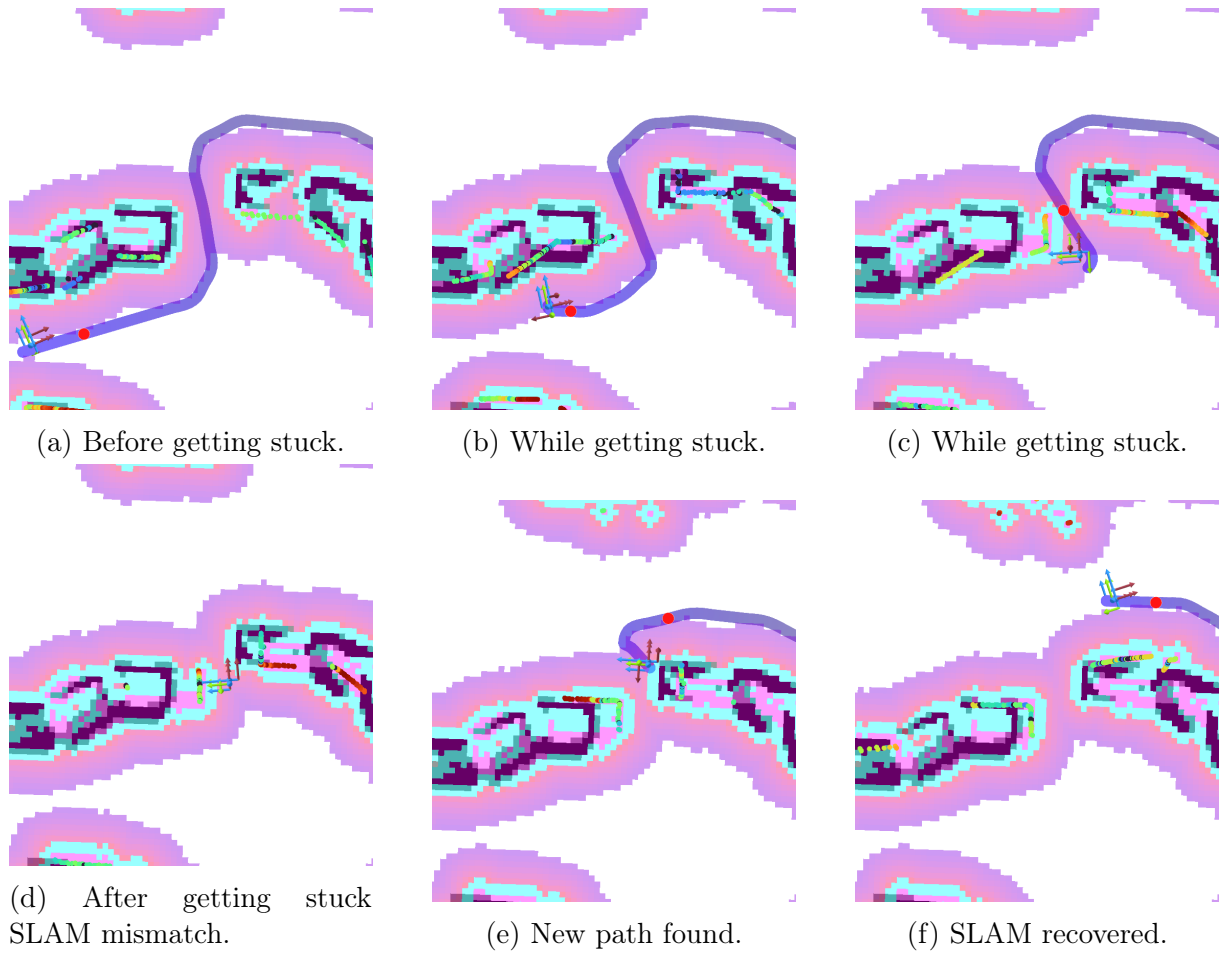


Figure 7: Issues with SLAM during drive from start S to goal I using theta star due to robot getting stuck on ground.

7 Further Improvements

In this section we would like to highlight problems that were beyond the scope of this project and ideas on how to tackle them.

When analysing the commanded linear and angular velocity `cmd_vel`, we noticed that the robot commands very high linear and angular jerk for all planners. This issue is due to the the planner, Pure Pursuit controller, switching controller behavior at the goal and abrupt stoppage due network connection loss. One can observe that A* which can only drive in octile directions and thus commands more turns, has larger root mean squared angular jerk then the other planners, see table 3.

The controller has no information or motivation to mitigate jerk. A simple solution could

	A*	A* Post-Processed	Theta*
RMS linear jerk (m/s^3)	45.0913	408.0230*	16.1919
RMS angular jerk (rad/s^3)	221.9241	202.3029	110.4150

Table 2: Jerk analysis. * the linear jerk for A* Post-Processed is significantly higher due to network connection loss during the experiment.

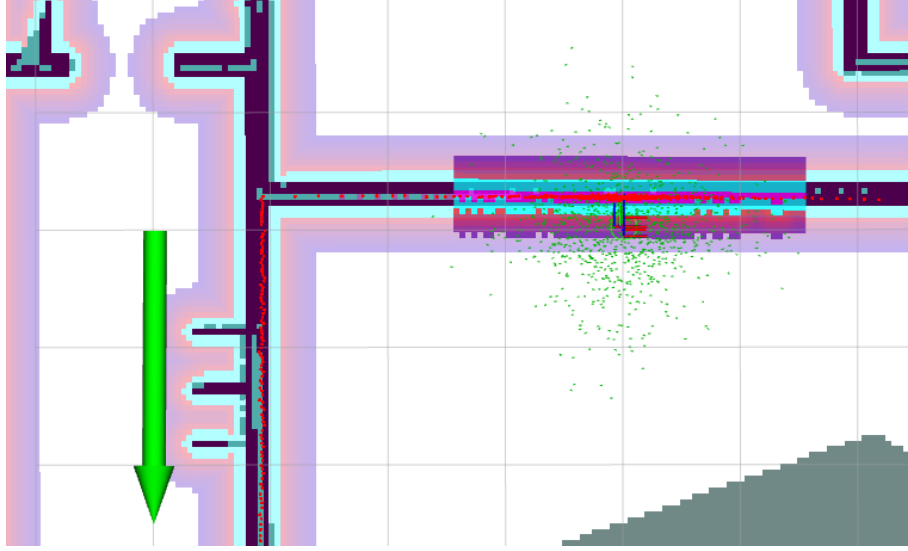


Figure 8: Setup of the experiments evaluating jerk. Image shows part of the map with starting and end position of the trajectory used for evaluation.

	Theta*	Theta* Rate-limited	Theta* First Order Filter $\alpha = 0.7$
RMS linear jerk (m/s^3)	6.0793	2.9737	3.5298
Max linear jerk (m/s^3)	99.3091	51.4674	52.5364
RMS angular jerk (rad/s^3)	17.5063	10.8092	10.7064
Max angular jerk (rad/s^3)	274.3988	175.0634	102.7737
Time to Goal s	93.9490	84.6090	77.9103

Table 3: Jerk analysis for different smoothing methods evaluated in simulation.

be to use Rate-limited commanded outputs algorithm 3 or use a First Order Low Pass Filter on the command outputs.

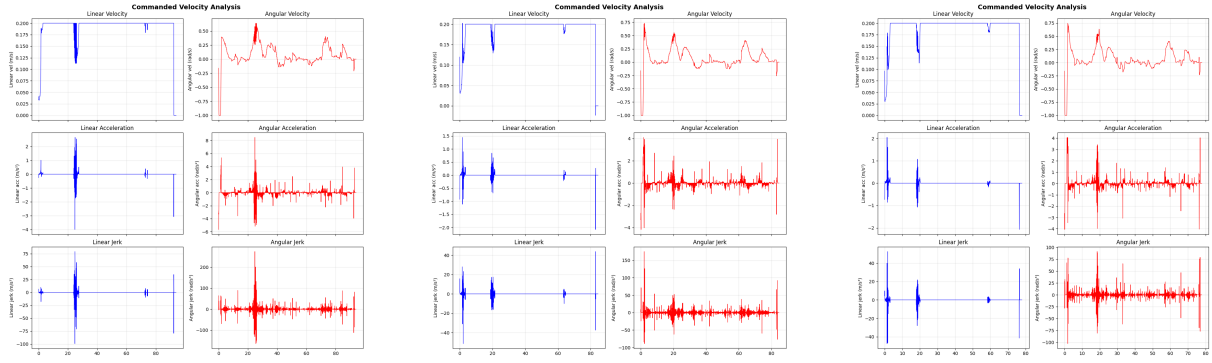
Algorithm 3 Acceleration Rate Limiting

Require: Desired velocity v_{des} , previous velocity v_{prev} , time step Δt , maximum acceleration a_{max}

Ensure: Rate-limited velocity v_{cmd}

- 1: $\Delta v \leftarrow v_{\text{des}} - v_{\text{prev}}$
 - 2: $\Delta v \leftarrow \text{clamp}(\Delta v, -a_{\text{max}} \cdot \Delta t, a_{\text{max}} \cdot \Delta t)$
 - 3: $v_{\text{cmd}} \leftarrow v_{\text{prev}} + \Delta v$
 - 4: **return** v_{cmd}
-

We were able to perform experiments in simulation, see fig. 8 and analyze the impact of Rate-limited commanded outputs and the use a First Order Filter, see fig. 9. We found that a simple low pass and the rate limiting filter have superior behavior in simulation. In fig. 9, the velocities, accelerations, and jerk profiles are plotted. Differences in the produced velocity profiles between the 3 methods are barely visible, but become very profound when looking at the jerk profiles. Both, the rate limiting and the low pass filter, roughly halve the RMS and maximum jerk compared to the original Theta*, table 3 while achieving better total time to goal.



(a) Without any smoothing or limiting of the command outputs.

(b) Rate-limited command outputs.

(c) First-Order low pass filter with $\alpha = 0.7$ for both angular and linear velocity.

Figure 9: Analysis for different smoothing methods evaluated in simulation.

Another problem, mentioned above, is the loss of connection to robot. This is serious problem and should be addressed. Our suggestion would be to not run controller not on the remote PC but run it instead on the Turtlebot itself while having the global path passed to the robot.

8 Contribution

Member	Contributions
A0337935R	• Regulated Pure Pursuit Implementation • A* Implementation • Foxglove Visualization
A0338361A	• Smoothing • Theta* Implementation • Parameter Tuning
A0338562X	• Regulated Pure Pursuit Implementation • Postprocessing A*: Waypoint Skipping

Table 4: Individual contributions.

References

- [1] Jack E. Bresenham. “Algorithm for computer control of a digital plotter”. In: *IBM Systems Journal* 4.1 (1965), pp. 25–30.
- [2] Foxglove Technologies. *Foxglove: Visualization and Observability for Robotics*. Accessed: 2026-02-26. 2021. URL: <https://foxglove.dev>.
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A formal basis for the heuristic determination of minimum cost paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.
- [4] N. Koenig and A. Howard. “Design and use paradigms for Gazebo, an open-source multi-robot simulator”. In: *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*. Vol. 3. 2004, 2149–2154 vol.3. DOI: [10.1109/IROS.2004.1389727](https://doi.org/10.1109/IROS.2004.1389727).
- [5] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: [10.1126/scirobotics.abm6074](https://doi.org/10.1126/scirobotics.abm6074). URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [6] Alex Nash et al. “Theta*: Any-Angle Path Planning on Grids”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2007, pp. 1177–1183.
- [7] Abraham Savitzky and Marcel J. E. Golay. “Smoothing and differentiation of data by simplified least squares procedures”. In: *Analytical Chemistry* 36.8 (1964), pp. 1627–1639.